

Recommendation System

A. Tổng quan

Có hai thực thể chính trong Recommendation Systems:

- + Users: người dùng.
- + Items: sản phẩm, ví dụ như các bộ phim, bài hát, cuốn sách, clip, hoặc cũng có thể là các users khác trong bài toán gợi ý kết bạn.

→ Mục đích chính của các Recommender Systems là dự đoán mức độ quan tâm của một user tới một item nào đó, qua đó có chiến lược recommend phù hợp.

Các Recommendation Systems thường được chia thành ba nhóm lớn:

- Content-based systems/filtering → features/properties of the contents/items: đánh giá đặc tính của items được recommended. Cách tiếp cận này yêu cầu việc sắp xếp các items vào từng nhóm hoặc đi tìm các đặc trưng của từng item. Tuy nhiên, có những items không có nhóm cụ thể và việc xác định nhóm hoặc đặc trưng của từng item đôi khi là bất khả thi.
- Collaborative filtering: hệ thống gợi ý items dựa trên sự tương quan (similarity) giữa các users và/hoặc items. Có thể hiểu rằng ở nhóm này một item được recommended tới một user dựa trên những users có hành vi tương tự. → how pp rate a product, gặp vấn đề cold start problem.
- Hybrid recommender system → both content-based and collaborative

Utility matrix

Mỗi user sẽ có mức độ quan tâm (degree of preference) tới từng item khác nhau. Mức độ quan tâm này, nếu đã biết trước, được gán cho một giá trị ứng với mỗi cặp user-item. Giả sử rằng mức độ quan tâm được đo bằng giá trị user rate cho item, ta tạm gọi giá trị này là rating. Tập hợp tất cả các ratings, bao gồm cả những giá trị chưa biết cần được dự đoán, tạo nên một ma trận gọi là utility matrix.

Thông thường, có rất nhiều users và items trong hệ thống, và mỗi user thường chỉ rate một số lượng rất nhỏ các item, thậm chí có những user không rate item nào (với những users này thì cách tốt nhất là gợi ý các items phổ biến nhất). Vì vậy, lượng ô màu xám của utility matrix trong các bài toán đó thường là rất lớn, và lượng các ô đã được điền là một số rất nhỏ.

Rõ ràng rằng càng nhiều ô được điền thì độ chính xác của hệ thống sẽ càng được cải thiện. Vì vậy, các hệ thống luôn luôn hỏi người dùng về sự quan tâm của họ tới sản phẩm, và muốn người dùng đánh giá càng nhiều sản phẩm càng tốt.

Có hai hướng tiếp cận phổ biến để xác định giá trị rating cho mỗi cặp user-item trong Utility Matrix:

- Nhờ người dùng rate sản phẩm bằng cách gửi các email nhắc nhở,... Tuy nhiên, cách tiếp cận này có một vài hạn chế, vì thường thì người dùng ít khi rate sản phẩm. Và nếu có, đó có thể là những đánh giá thiên lệch bởi những người sẵn sàng rate.
- Hướng tiếp cận thứ hai là dựa trên hành vi của users. Thường thì với cách này, ta chỉ xây dựng được một ma trận với các thành phần là 1 và 0, với 1 thể hiện người dùng thích sản phẩm, 0 thể hiện chưa có thông tin. Trong trường hợp này, 0 không có nghĩa là thấp hơn 1, nó chỉ có nghĩa là người dùng chưa cung cấp thông tin. Chúng ta cũng có thể xây dựng ma trận với các giá trị cao hơn 1 thông qua thời gian hoặc số lượt mà người dùng xem một sản phẩm nào đó. Đôi khi, nút dislike cũng mang lại những lợi ích nhất định cho hệ thống, lúc này có thể gán giá trị tương ứng bằng -1 chẳng hạn.

B. Content-based filtering

Item profiles

Trong các hệ thống content-based, tức dựa trên nội dung của mỗi item, chúng ta cần xây dựng một bộ hồ sơ (profile) cho mỗi item. Profile này được biểu diễn dưới dạng toán học là một feature vector. Trong những trường hợp đơn giản, feature vector được trực tiếp trích xuất từ item.

Bài toán tìm mô hình

Ở đây cho mỗi user có thể được coi là một bài toán Regression trong trường hợp ratings là một dải giá trị, hoặc bài toán Classification trong trường hợp ratings là một vài trường hợp cụ thể, như like/dislike chẳng hạn. Dữ liệu training để xây dựng mỗi mô hình ở đây là các cặp (item profile, ratings) tương ứng với các items mà user đó đã rated. Việc điền các giá trị còn thiếu trong ma trận Utility chính là việc dự đoán đầu ra cho các unrated items khi áp dụng mô hình ở đây lên chúng.

Việc lựa chọn mô hình Regression/Classification nào tùy thuộc vào ứng dụng. Ví dụ về một mô hình đơn giản nhất: mô hình tuyến tính, mà cụ thể là Linear Regression với regularization, tức Ridge Regression.

Xây dựng hàm mất mát

Giả sử rằng số users là N , số items là M , utility matrix được mô tả bởi ma trận Y .

Thành phần ở hàng thứ m , cột thứ n của Y là mức độ quan tâm (ở đây là số sao đã rate) của user thứ n lên sản phẩm thứ m mà hệ thống đã thu thập được. Ma trận Y bị khuyết rất nhiều thành phần tương ứng với các giá trị mà hệ thống cần dự đoán. Thêm nữa, gọi R là ma trận rated or not thể hiện việc một user đã rated một item hay chưa. Cụ thể, r_{ij} bằng 1 nếu item thứ i đã được rated bởi user thứ j , bằng 0 trong trường hợp ngược lại.

Mô hình tuyến tính: Giả sử rằng ta có thể tìm được một mô hình cho mỗi user, minh hoạ bởi vector cột hệ số $\mathbf{w}_i$ và bias b_n sao cho mức độ quan tâm của một user tới một item có thể tính được bằng một hàm tuyến tính:

Giả sử rằng ta có thể tìm được một mô hình cho mỗi *user*, minh hoạ bởi vector cột hệ số $\mathbf{w}_i$ và bias b_n sao cho mức độ quan tâm của một *user* tới một *item* có thể tính được bằng một hàm tuyến tính:

$$y_{mn} = \mathbf{x}_m \mathbf{w}_n + b_n \quad (1)$$

(Chú ý rằng $\mathbf{x}_m$ là một vector hàng, $\mathbf{w}_n$ là một vector cột.)

Xét một *user* thứ n bất kỳ, nếu ta coi training set là tập hợp các thành phần đã được *điền* của $\mathbf{y}_n$, ta có thể xây dựng hàm mất mát tương tự như Ridge Regression như sau:

$$\mathcal{L}_n = \frac{1}{2} \sum_{m: r_{mn}=1} (\mathbf{x}_m \mathbf{w}_n + b_n - y_{mn})^2 + \frac{\lambda}{2} \|\mathbf{w}_n\|_2^2$$

Trong đó, thành phần thứ hai là regularization term và λ là một tham số dương. Chú ý rằng regularization thường không được áp dụng lên bias b_n . Trong thực hành, trung bình cộng của *lỗi* thường được dùng, và *mất mát* $\mathcal{L}_n$ được viết lại thành:

$$\mathcal{L}_n = \frac{1}{2s_n} \sum_{m: r_{mn}=1} (\mathbf{x}_m \mathbf{w}_n + b_n - y_{mn})^2 + \frac{\lambda}{2s_n} \|\mathbf{w}_n\|_2^2$$

Trong đó s_n là số lượng các *items* mà *user* thứ n đã *rated*. Nói cách khác:

$$s_n = \sum_{m=1}^M r_{mn},$$

là tổng các phần tử trên cột thứ n của ma trận *rated or not* $\mathbf{R}$.

Vì biểu thức *loss function* chỉ phụ thuộc vào các *items* đã được *rated*, ta có thể rút gọn nó bằng cách đặt $\hat{\mathbf{y}}_n$ là *sub vector* của $\mathbf{y}$ được xây dựng bằng cách *trích* các thành phần *khác dấu* ? ở cột thứ n , tức đã được *rated* bởi *user* thứ n trong Utility Matrix $\mathbf{Y}$. Đồng thời, đặt $\hat{\mathbf{X}}_n$ là *sub matrix* của ma trận feature $\mathbf{X}$, được tạo bằng cách *trích* các hàng tương ứng với các *items* đã được *rated* bởi *user* thứ n . (Xem ví dụ phía dưới để hiểu rõ hơn). Khi đó, biểu thức hàm mất mát của mô hình cho *user* thứ n được viết gọn thành:

$$\mathcal{L}_n = \frac{1}{2s_n} \|\hat{\mathbf{X}}_n \mathbf{w}_n + b_n \mathbf{e}_n - \hat{\mathbf{y}}_n\|_2^2 + \frac{\lambda}{2s_n} \|\mathbf{w}_n\|_2^2$$

trong đó, $\mathbf{e}_n$ là vector cột chứa s_n phần tử 1.

Đây *chính xác* là hàm mất mát của Ridge Regression. Cặp nghiệm $\mathbf{w}_n, b_n$ có thể được tìm qua Stochastic Gradient Descent (SGD), hoặc Mini-batch GD. Tôi không đi sâu vào việc tính đạo hàm theo $\mathbf{w}_n$ và b_n của $\mathcal{L}_n$ nữa. Việc này đã được đề cập nhiều trong các bài trước. Trong bài này, tôi sẽ sử dụng class `Ridge` trong `sklearn.linear_model`.

Ưu điểm:

- Tiết kiệm bộ nhớ và thời gian tính toán.
- Đồng thời, hệ thống có khả năng tận dụng các thông tin đặc trưng của mỗi item như được mô tả trong bản mô tả (description) của mỗi item. Bản mô tả này có thể được xây dựng bởi nhà cung cấp hoặc được thu thập bằng cách yêu cầu users gắn tags cho items.
- Không bị ảnh hưởng bởi user xấu (fake reviews)

Nhược điểm:

- Không tận dụng được thông tin từ các users khác. Những thông tin này thường rất hữu ích vì hành vi mua hàng của các users thường được nhóm thành một vài nhóm đơn giản; nếu biết hành vi mua hàng của một vài users trong nhóm, hệ thống nên suy luận ra hành vi của những users còn lại.
- Không phải lúc nào cũng có bản mô tả (metadata) cho mỗi item. Việc yêu cầu users gắn tags còn khó khăn hơn vì không phải ai cũng sẵn sàng làm việc đó; hoặc có làm nhưng sẽ mang xu hướng cá nhân.
- Không khám phá được sở thích mới, chỉ toàn gợi ý phim "giống cũ". User thích hành động mãi chỉ nhận được phim hành động
- Cold start cho item mới → Phim mới chưa có đủ metadata → không gợi ý được

Các bước chính:

Bước 1 - Xây dựng Item Profile: Mỗi phim được biểu diễn bằng 1 vector số thể hiện đặc trưng của nó.

```
import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import TfidfTransformer
```

Giả sử có bảng metadata phim với các thể loại

Mỗi hàng = 1 phim, mỗi cột = 1 thể loại (0/1)

	Hành_động	Lãng_mạn	Hoạt_hình	Hài	Kinh_dị
Đấu Phá	[1	0	0	0	0]
Mị Nguyệt	[1	1	0	0	0]
Doraemon	[0	0	1	1	0]

```
X_counts = movie_info_matrix
```

 ma trận 0/1 thể loại phim

Chuyển sang TF-IDF

smooth_idf=True: tránh lỗi khi thể loại không có phim nào

norm='l2': chuẩn hóa để mọi vector có độ dài = 1

```
transformer = TfidfTransformer(smooth_idf=True, norm='l2')
tfidf = transformer.fit_transform(X_counts).toarray()
```

```
print(f"Item profile shape: {tfidf.shape}")
```

→ (số_phim × số_thể_loại)

Ví dụ: (1682, 19) với MovieLens

Trước và sau TF-IDF:

Trước (0/1):

Sau (TF-IDF):

Hành_động = 1 → Hành_động = 0.28 (phổ biến → thấp)

Lãng_mạn = 1 → Lãng_mạn = 0.71 (ít hơn → cao hơn)

Hoạt_hình = 0 → Hoạt_hình = 0.00

Bước 2 - Xây dựng Utility Matrix: Bảng lưu ai đã xem phim nào và cho điểm bao nhiêu.

Utility Matrix Y (M phim $\times$ N users)

Ô đã biết = rating thật

Ô trống = cần dự đoán

Ví dụ minh họa:

	User_A	User_B	User_C	User_D
Đấu Phá	[5	4	?	?
Mị Nguyệt	[4	5	?	?
Doraemon	[?	?	5	4
Conan	[?	?	4	5

Trong thực tế với MyTV:

737,265 users $\times$ 3,936 phim

Chỉ có 6,466,174/~2.9 tỷ ô có giá trị (~0.22%)

→ Ma trận rất thưa (sparse)

Implicit rating từ duration (vì MyTV không có rating thật):

interaction = (

```
df.groupby(['member_id', 'content_id'])['duration']
```

```
.sum()
```

```
.reset_index()
```

```
)
```

```
interaction['score'] = np.log1p(interaction['duration'])
```

log1p giúp giảm chênh lệch giữa xem ít và xem nhiều

```
print(f"Số interactions: {len(interaction):,}")
```

```
print(f"Score range: {interaction['score'].min():.2f} →
```

```
{interaction['score'].max():.2f}")
```

Bước 3 - Train model cho từng user: Mỗi user có 1 mô hình riêng thể hiện sở thích của họ.

```
from sklearn.linear_model import Ridge
```

```
def get_itemsRated_by_user(rate_matrix, user_id):  
    """Lấy phim đã xem và điểm của 1 user"""  
    ids = np.where(rate_matrix[:, 0] == user_id)[0]  
    item_ids = rate_matrix[ids, 1].astype(int)  
    scores = rate_matrix[ids, 2]  
    return item_ids, scores
```

```
n_features = tfidf.shape[1] # số thể loại phim  
n_users = ... # số users
```

Khởi tạo ma trận lưu sở thích của tất cả users

```
W = np.zeros((n_features, n_users)) # (thể_loại × users)  
b = np.zeros((1, n_users)) # bias
```

```
for n in range(n_users):
```

Lấy phim đã xem của user n

```
item_ids, scores = get_itemsRated_by_user(rate_train, n)
```

```
if len(item_ids) == 0:
```

```
    continue # bỏ qua user chưa xem phim nào
```

$\hat{X}_n$ = feature của những phim user n đã xem

```
Xhat = tfidf[item_ids, :]
```

Ridge Regression: tìm w_n và b_n tốt nhất

$\alpha = \lambda$ = regularization $\rightarrow$ tránh overfitting

```
clf = Ridge(alpha=0.01, fit_intercept=True)
```

```
clf.fit(Xhat, scores)
```

Lưu kết quả

```
W[:, n] = clf.coef_ # sở thích của user n
```

```
b[0, n] = clf.intercept_ # bias của user n
```

Ý nghĩa của $W[:, n]$:

$w_n = [3.2, 0.8, 0.1, \dots]$

$\rightarrow$ user n thích thể loại 1 (hành động) = 3.2

$\rightarrow$ thích thể loại 2 (lãng mạn) = 0.8

→ thích thể loại 3 (hoạt hình) = 0.1

Bước 4 - Dự đoán score: Điền vào toàn bộ Utility Matrix → biết user sẽ cho phim nào bao nhiêu điểm.

Dự đoán TẤT CẢ cặp (phim, user) trong 1 dòng code

```
Yhat = tfidf.dot(W) + b
```

tfidf shape: (M_phim × K_thể_loại)

W shape: (K_thể_loại × N_users)

Yhat shape: (M_phim × N_users)

→ $\hat{O}[m, n]$ = score dự đoán của user n cho phim m

Ví dụ xem kết quả với 1 user:

```
user_id = 10
```

```
item_ids, true_scores = get_itemsRatedByUser(rate_test, user_id)
```

```
print("Phim đã xem:", item_ids)
```

```
print("Điểm thật :", true_scores)
```

```
print("Điểm dự đoán:", Yhat[item_ids, user_id].round(2))
```

Kết quả ví dụ:

Điểm thật : [3 3 4 3 5 3]

Điểm dự đoán: [3.18 3.13 3.42 3.09 4.01 3.35]

Bước 5 - Đánh giá RMSE: Đo độ chính xác của model - sai trung bình bao nhiêu điểm.

```
from math import sqrt
```

```
def evaluate_rmse(Yhat, rate_data, n_users):
```

```
    """Tính RMSE trên tập dữ liệu"""
```

```
    se = 0 # tổng bình phương sai số
```

```
    cnt = 0 # tổng số ratings
```

```
    for n in range(n_users):
```

```
        item_ids, scores_truth = get_itemsRatedByUser(rate_data, n)
```

```
        if len(item_ids) == 0:
```

```
            continue
```

```
        scores_pred = Yhat[item_ids, n] # lấy điểm dự đoán
```

```

error    = scores_truth - scores_pred # tính sai số
se += (error ** 2).sum()              # cộng bình phương
cnt += len(error)                     # đếm số ratings

```

```

return sqrt(se / cnt)

```

```

rmse_train = evaluate_rmse(Yhat, rate_train, n_users)
rmse_test  = evaluate_rmse(Yhat, rate_test, n_users)

```

```

print(f"RMSE train: {rmse_train:.4f}") # → 0.9090
print(f"RMSE test : {rmse_test:.4f}")  # → 1.2703

```

Đọc kết quả:

RMSE = 1.27 trên thang điểm 1-5
 → sai trung bình 1.27 điểm
 → chưa tốt vì chỉ dùng thể loại phim

Bước 6 - Gợi ý top N phim: Từ Yhat đã có → lọc phim chưa xem → sort theo điểm
 → lấy top N.

```

def recommend_top_n(user_id, Yhat, rate_train, n_items, top_n=10):
    """Gợi ý top N phim cho 1 user"""

```

Lấy phim đã xem của user

```

watched_ids, _ = get_itemsRated_by_user(rate_train, user_id)
watched_set    = set(watched_ids)

```

Lấy điểm dự đoán cho tất cả phim

```

all_scores = Yhat[:, user_id]

```

Chỉ lấy phim chưa xem

```

candidates = [
    (item_id, all_scores[item_id])
    for item_id in range(n_items)
    if item_id not in watched_set
]

```

Sort theo điểm giảm dần

```

candidates.sort(key=lambda x: x[1], reverse=True)

```

```
return candidates[:top_n]
```

Ví dụ gợi ý cho user 10:

```
top10 = recommend_top_n(  
    user_id=10,  
    Yhat=Yhat,  
    rate_train=rate_train,  
    n_items=tfidf.shape[0],  
    top_n=10  
)
```

```
print("Top 10 phim gợi ý cho user 10:")
```

```
for item_id, score in top10:
```

```
    print(f"Phim {item_id}: điểm dự đoán = {score:.2f}")
```

C. Collaborative Filtering (CF)

- **Neighborhood-based Collaborative Filtering (NBCF).**

Ý tưởng cơ bản của NBCF là xác định mức độ quan tâm của một user tới một item dựa trên các users khác gần giống với user này. Việc gần giống nhau giữa các users có thể được xác định thông qua mức độ quan tâm của các users này tới các items khác mà hệ thống đã biết.

→ Hai câu hỏi quan trọng nhất:

- Làm thế nào xác định được sự giống nhau giữa hai users?
- Khi đã xác định được các users gần giống nhau (similar users) rồi, làm thế nào dự đoán được mức độ quan tâm của một user lên một item?

Việc xác định mức độ quan tâm của mỗi user tới một item dựa trên mức độ quan tâm của similar users tới item đó còn được gọi là User-user collaborative filtering.

a. User-user collaborative filtering

Công việc quan trọng nhất phải làm trước tiên trong User-user Collaborative Filtering là phải xác định được sự giống nhau (similarity) giữa hai users. Dữ liệu duy nhất chúng ta có là Utility matrix Y , vậy nên sự giống nhau này phải được xác định dựa trên các cột tương ứng với hai users trong ma trận này. Xét ví dụ trong Hình 1.

	u_0	u_1	u_2	u_3	u_4	u_5	u_6
i_0	5	5	2	0	1	?	?
i_1	3	?	?	0	?	?	?
i_2	?	4	1	?	?	1	2
i_3	2	2	3	4	4	?	4
i_4	2	0	4	?	?	?	5

Hình 1: Ví dụ về utility matrix dựa trên số sao một *user rate* cho một *item*. Một cách trực quan, *hành vi* của u_0 giống với u_1 hơn là u_2, u_3, u_4, u_5, u_6 . Từ đó có thể dự đoán rằng u_0 sẽ quan tâm tới i_2 vì u_1 cũng quan tâm tới *item* này.

Giả sử có các *users* từ u_0 đến u_6 và các *items* từ i_0 đến i_4 trong đó các số trong mỗi ô vuông thể hiện số sao mà mỗi *user* đã *rated* cho *item* với giá trị cao hơn thể hiện *mức độ quan tâm* cao hơn. Các dấu hỏi chấm là các giá trị mà hệ thống cần phải đi tìm. Đặt *mức độ giống nhau* của hai *users* u_i, u_j là $\text{sim}(u_i, u_j)$.

Quan sát đầu tiên chúng ta có thể nhận thấy là các u_0, u_1 *thích* i_0, i_1, i_2 và *không thích* i_3, i_4 cho lắm. Điều ngược lại xảy ra ở các *users* còn lại. Vì vậy, một *similarity function* tốt cần đảm bảo:

$$\text{sim}(u_0, u_1) > \text{sim}(u_0, u_i), \forall i > 1.$$

Từ đó, để xác định *mức độ quan tâm* của u_0 lên i_2 , chúng ta nên dựa trên *hành vi* của u_1 lên sản phẩm này. Rất may rằng u_1 đã *thích* i_2 nên hệ thống cần *recommend* i_2 cho u_0 .

Để đo similarity giữa hai *users*, cách thường làm là xây dựng feature vector cho mỗi *user* rồi áp dụng một hàm có khả năng đo similarity giữa hai vectors đó.

Các vectors này được xây dựng trực tiếp dựa trên Utility matrix chứ không dùng dữ liệu ngoài như item profiles.

Vậy mỗi dấu ‘?’ nên được thay bởi giá trị nào để hạn chế việc sai lệch quá nhiều? Một giá trị khả dĩ hơn cho việc này là trung bình cộng của các ratings mà *user* tương ứng đã thực hiện. Việc này sẽ tránh được việc *users* quá khó tính hoặc dễ tính, tức lúc nào cũng có những *items* mà một *user* thích hơn so với những *items* khác.

	u_0	u_1	u_2	u_3	u_4	u_5	u_6
i_0	5	5	2	0	1	?	?
i_1	4	?	?	0	?	?	?
i_2	?	4	1	?	?	1	1
i_3	2	2	3	4	4	?	4
i_4	2	0	4	?	?	?	5
$\bar{u}_j$	3.25	2.75	2.5	1.33	2.5	1.5	3.33

a) Original utility matrix $\mathbf{Y}$ and mean user ratings.

	u_0	u_1	u_2	u_3	u_4	u_5	u_6
i_0	1.75	2.25	-0.5	-1.33	-1.5	0	0
i_1	0.75	0	0	-1.33	0	0.5	0
i_2	0	1.25	-1.5	0	0	-0.5	-2.33
i_3	-1.25	-0.75	0.5	2.67	1.5	0	0.67
i_4	-1.25	-2.75	1.5	0	0	0	1.67

b) Normalized utility matrix $\tilde{\mathbf{Y}}$.

	u_0	u_1	u_2	u_3	u_4	u_5	u_6
u_0	1	0.83	-0.58	-0.79	-0.82	0.2	-0.38
u_1	0.83	1	-0.87	-0.40	-0.55	-0.23	-0.71
u_2	-0.58	-0.87	1	0.27	0.32	0.47	0.96
u_3	-0.79	-0.40	0.27	1	0.87	-0.29	0.18
u_4	-0.82	-0.55	0.32	0.87	1	0	0.16
u_5	0.2	-0.23	0.47	-0.29	0	1	0.56
u_6	-0.38	-0.71	0.96	0.18	0.16	0.56	1

c) User similarity matrix $\mathbf{S}$.

	u_0	u_1	u_2	u_3	u_4	u_5	u_6
i_0	1.75	2.25	-0.5	-1.33	-1.5	0.18	-0.63
i_1	0.75	0.48	-0.17	-1.33	-1.33	0.5	0.05
i_2	0.91	1.25	-1.5	-1.84	-1.78	-0.5	-2.33
i_3	-1.25	-0.75	0.5	2.67	1.5	0.59	0.67
i_4	-1.25	-2.75	1.5	1.57	1.56	1.59	1.67

d) $\tilde{\mathbf{Y}}$

Predict normalized rating of u_1 on i_1 with $k = 2$

Users who rated i_1 : $\{u_0, u_3, u_5\}$

Corresponding similarities: $\{0.83, -0.40, -0.23\}$

$\Rightarrow$ most similar users: $\mathcal{N}(u_1, i_1) = \{u_0, u_5\}$

with **normalized ratings** $\{0.75, 0.5\}$

$$\Rightarrow \hat{y}_{i_1, u_1} = \frac{0.83 \cdot 0.75 + (-0.23) \cdot 0.5}{0.83 + |-0.23|} \approx 0.48$$

e) Example

	u_0	u_1	u_2	u_3	u_4	u_5	u_6
i_0	5	5	2	0	1	1.68	2.70
i_1	4	3.23	2.33	0	1.67	2	3.38
i_2	4.15	4	1	-0.5	0.71	1	1
i_3	2	2	3	4	4	2.10	4
i_4	2	0	4	2.9	4.06	3.10	5

f) Full $\mathbf{Y}$

Hình 2: Ví dụ mô tả User-user Collaborative Filtering. a) Utility Matrix ban đầu. b) Utility Matrix đã được chuẩn hoá. c) User similarity matrix. d) Dự đoán các (normalized) *ratings* còn thiếu. e) Ví dụ về cách dự đoán normalized rating của u_1 cho i_1 . f) Dự đoán các (denormalized) *ratings* còn thiếu.

Chuẩn hóa dữ liệu

Hàng cuối cùng trong Hình 2a) là giá trị trung bình của ratings cho mỗi user. Giá trị cao tương ứng với các user dễ tính và ngược lại. Khi đó, nếu tiếp tục trừ từ mỗi rating đi giá trị này và thay các giá trị chưa biết bằng 0, ta sẽ được normalized utility matrix như trong Hình 2b).

- Việc trừ đi trung bình cộng của mỗi cột khiến trong mỗi cột có những giá trị dương và âm.
- Những giá trị dương tương ứng với việc user thích item, những giá trị âm tương ứng với việc user không thích item.
- Những giá trị bằng 0 tương ứng với việc chưa xác định được liệu user có thích item hay không.

Cosine Similarity

$$\text{cosine_similarity}(\mathbf{u}_1, \mathbf{u}_2) = \cos(\mathbf{u}_1, \mathbf{u}_2) = \frac{\mathbf{u}_1^T \mathbf{u}_2}{\|\mathbf{u}_1\|_2 \cdot \|\mathbf{u}_2\|_2} \quad (1)$$

Trong đó $\mathbf{u}_{1,2}$ là vectors tương ứng với *users* 1, 2 **đã được chuẩn hoá** như ở trên.

Có một tin vui là python có hàm hỗ trợ tính toán hàm số này một cách hiệu quả.

Độ *similarity* của hai vector là 1 số trong đoạn $[-1, 1]$. Giá trị bằng 1 thể hiện hai vector hoàn toàn *similar* nhau. Hàm số cos của một góc bằng 1 nghĩa là góc giữa hai vector bằng 0, tức một vector bằng tích của một số dương với vector còn lại. Giá trị cos bằng -1 thể hiện hai vector này hoàn toàn trái ngược nhau. Điều này cũng hợp lý, tức khi *hành vi* của hai *users* là hoàn toàn ngược nhau thì *similarity* giữa hai vector đó là thấp nhất.

Ví dụ về cosine_similarity của các *users* trong Hình 2b) được cho trong Hình 2c). Similarity matrix **S** là một ma trận đối xứng vì cos là một hàm chẵn, và nếu *user A giống user B* thì điều ngược lại cũng đúng. Các ô màu xanh trên đường chéo đều bằng 1 vì đó là cos của góc giữa 1 vector và chính nó, tức $\cos(0) = 1$. Khi tính toán ở các bước sau, chúng ta không cần quan tâm tới các giá trị 1 này. Tiếp tục quan sát các vector hàng tương ứng với u_0, u_1, u_2 , chúng ta sẽ thấy một vài điều thú vị:

- u_0 gần với u_1 và u_5 (độ giống nhau là dương) hơn các *users* còn lại. Việc *similarity* cao giữa u_0 và u_1 là dễ hiểu vì cả hai đều có xu hướng quan tâm tới i_0, i_1, i_2 hơn các *items* còn lại. Việc u_0 gần với u_5 thoạt đầu có vẻ vô lý vì u_5 đánh giá thấp các *items* mà u_0 đánh giá cao (Hình 2a)); tuy nhiên khi nhìn vào ma trận utility đã chuẩn hoá ở Hình 2b), ta thấy rằng điều này là hợp lý. Vì *item* duy nhất mà cả hai *users* này đã cung cấp thông tin là i_1 với các giá trị tương ứng đều là *tích cực*.
- u_1 gần với u_0 và xa các *users* còn lại.
- u_2 gần với u_3, u_4, u_5, u_6 và xa các *users* còn lại.

Từ similarity matrix này, chúng ta có thể phân nhóm các users ra làm hai nhóm (u_0, u_1) và (u_2, u_3, u_4, u_5, u_6). Vì ma trận S này nhỏ nên chúng ta có thể dễ dàng quan sát thấy điều này; khi số users lớn hơn, việc xác định bằng mắt thường là không khả thi. Việc xây dựng thuật toán phân nhóm các users (users clustering) rất có thể sẽ được trình bày ở một trong các bài viết tiếp theo.

Có một chú ý quan trọng ở đây là khi số lượng users lớn, ma trận S cũng rất lớn và nhiều khả năng là không có đủ bộ nhớ để lưu trữ, ngay cả khi chỉ lưu hơn một nửa số các phần tử của ma trận đối xứng này. Với các trường hợp đó, mới mỗi user, chúng ta chỉ cần tính và lưu kết quả của một hàng của similarity matrix, tương ứng với việc độ giống nhau giữa user đó và các users còn lại.

Rating Prediction

Xác định mức độ quan tâm của một user lên một item dựa trên các users gần nhất (neighbor users).

Tương tự như KNN, trong Collaborative Filtering, *missing rating* cũng được xác định dựa trên thông tin về k neighbor users. Tất nhiên, chúng ta chỉ quan tâm tới **các users đã rated item đang xét**. *Predicted rating* thường được xác định là *trung bình có trọng số* của các *ratings đã chuẩn hoá*. Có một điểm cần lưu ý, trong KNN, các trọng số được xác định dựa trên *distance* giữa 2 điểm, và các *distance* này là các số không âm. Trong khi đó, trong CF, các trọng số được xác định dựa trên *similarity* giữa hai users, những trọng số này có thể nhỏ hơn 0 như trong Hình 2c).

Công thức phổ biến được sử dụng để dự đoán *rating* của u cho i là:

$$\hat{y}_{i,u} = \frac{\sum_{u_j \in \mathcal{N}(u,i)} \bar{y}_{i,u_j} \text{sim}(u, u_j)}{\sum_{u_j \in \mathcal{N}(u,i)} |\text{sim}(u, u_j)|} \quad (2)$$

(sự khác biệt so với trung bình có trọng số là mẫu số có sử dụng trị tuyệt đối để xử lý các số âm).

trong đó $\mathcal{N}(u, i)$ là tập hợp k users trong *neighborhood* (tức có *similarity* cao nhất) của u mà **đã rated i** .

Hình 2d) thể hiện việc *điền* các giá trị còn thiếu trong *normalized utility matrix*. Các ô màu nền đỏ thể hiện các giá trị dương, tức các *items* mà *có thể* users đó *quan tâm*. Ở đây, tôi đã lấy ngưỡng bằng 0, chúng ta hoàn toàn có thể chọn các *ngưỡng* khác 0.

Một ví dụ về việc tính *normalized rating* của u_1 cho i_1 được cho trong Hình 2e) với số *nearest neighbors* là $k = 2$. Các bước thực hiện là:

1. Xác định các users đã *rated* i_1 , đó là u_0, u_3, u_5 .
2. Xác định *similarities* của u_1 với các users này ta nhận được 0.83, -0.40 , -0.23 . Hai ($k = 2$) giá trị lớn nhất là 0.83 và -0.23 tương ứng với u_0 và u_5 .
3. Xác định các *normalized ratings* của u_0, u_5 cho i_1 , ta thu được hai giá trị lần lượt là 0.75 và 0.5.
4. Dự đoán kết quả:

$$\hat{y}_{i_1, u_1} = \frac{0.83 \times 0.75 + (-0.23) \times 0.5}{0.83 + |-0.23|} \approx 0.48$$

Việc quy đổi các giá trị ratings đã chuẩn hoá về thang 5 có thể được thực hiện bằng cách cộng các cột của ma trận $\hat{Y}$ với giá trị rating trung bình của mỗi user như đã tính trong Hình 2a).

Việc hệ thống quyết định *recommend items* nào cho mỗi user có thể được xác định bằng nhiều cách khác nhau. Có thể sắp xếp *unrated items* theo thứ tự từ lớn đến bé của các *predicted ratings*, hoặc chỉ chọn các *items* có *normalized predicted ratings* dương - tương ứng với việc user này có nhiều khả năng thích hơn.

Trước khi vào phần lập trình cho User-user CF, chúng ta cùng xem xét Item-item CF.

b. Item-item collaborative filtering

Một số hạn chế của User-user CF:

- Trên thực tế, số lượng users luôn lớn hơn số lượng items rất nhiều. Kéo theo đó là Similarity matrix là rất lớn với số phần tử phải lưu giữ là hơn 1 nửa của bình phương số lượng users (chú ý rằng ma trận này là đối xứng). Việc này, như đã đề cập ở trên, khiến cho việc lưu trữ ma trận này trong nhiều trường hợp là không khả thi.
- Ma trận Utility Y thường là rất sparse. Với số lượng users rất lớn so với số lượng items, rất nhiều cột của ma trận này sẽ rất sparse, tức chỉ có một vài phần tử khác 0. Lý do là users thường lười rating. Cũng chính vì việc này, một khi user đó thay đổi rating hoặc rate thêm items, trung bình cộng các ratings cũng

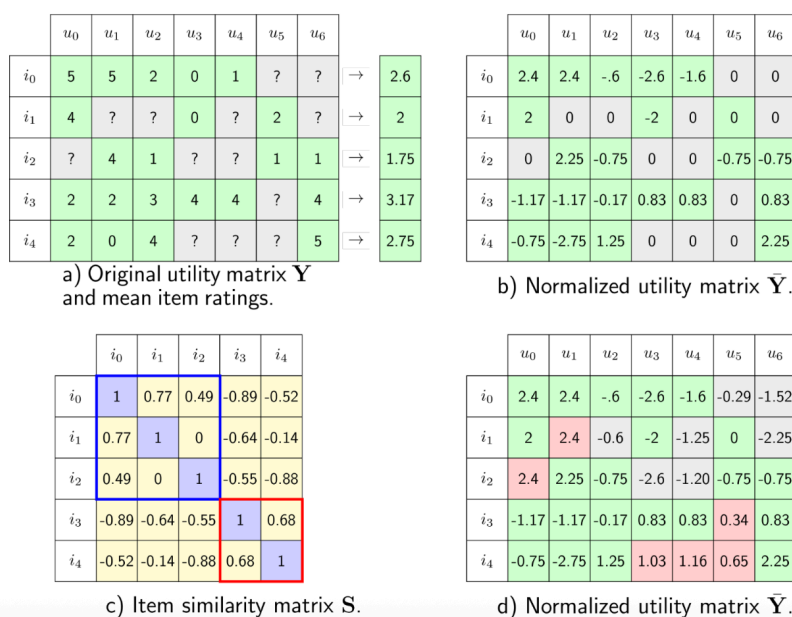
như vector chuẩn hóa tương ứng với user này thay đổi nhiều. Kéo theo đó, việc tính toán ma trận Similarity, vốn tốn nhiều bộ nhớ và thời gian, cũng cần được thực hiện lại.

Ngược lại, nếu chúng ta tính toán similarity giữa các items rồi recommend những items gần giống với item yêu thích của một user thì sẽ có những lợi ích sau:

- Vì số lượng items thường nhỏ hơn số lượng users, Similarity matrix trong trường hợp này cũng nhỏ hơn nhiều, thuận lợi cho việc lưu trữ và tính toán ở các bước sau.
- Vì số lượng phần tử đã biết trong Utility matrix là như nhau nhưng số hàng (items) ít hơn số cột (users), nên trung bình, mỗi hàng của ma trận này sẽ có nhiều phần tử đã biết hơn số phần tử đã biết trong mỗi cột. Việc này cũng dễ hiểu vì mỗi item có thể được rated bởi nhiều users. Kéo theo đó, giá trị trung bình của mỗi hàng ít bị thay đổi hơn khi có thêm một vài ratings. Như vậy, việc cập nhật ma trận Similarity Matrix có thể được thực hiện ít thường xuyên hơn.

Cách tiếp cận thứ hai này được gọi là Item-item Collaborative Filtering. Hướng tiếp cận này được sử dụng nhiều trong thực tế hơn.

Quy trình dự đoán missing ratings cũng tương tự như trong User-user CF. Hình 3 mô tả quy trình này với ví dụ nêu ở phần trên.



Hình 3: Ví dụ mô tả Item-Item Collaborative Filtering. a) Utility Matrix ban đầu. b) Utility Matrix đã được chuẩn hoá. c) User similarity matrix. d) Dự đoán các (normalized) ratings còn thiếu.

Có một điểm thú vị trong Similarity matrix ở Hình 3c) là có các phần tử trong hai hình vuông xanh và đỏ đều là các số không âm, các phần tử bên ngoài là các số âm. Việc này thể hiện rằng các items có thể được chia thành 2 nhóm rõ rệt với những items có

similarity không âm vào 1 nhóm. Như vậy, một cách vô tình, chúng ta đã thực hiện việc item clustering. Việc này sẽ giúp ích rất nhiều trong việc dự đoán ở phần sau.

Ma trận đầy đủ (tồn RAM):

```
[[0, 0, 5, 0, 0, 0, 0, 4, 0, 0],  
 [0, 3, 0, 0, 0, 0, 0, 0, 0, 0],  
 ...]
```

Sparse matrix (chỉ lưu ô khác 0):

```
[(0,2,5), (0,7,4), (1,1,3), ...]
```

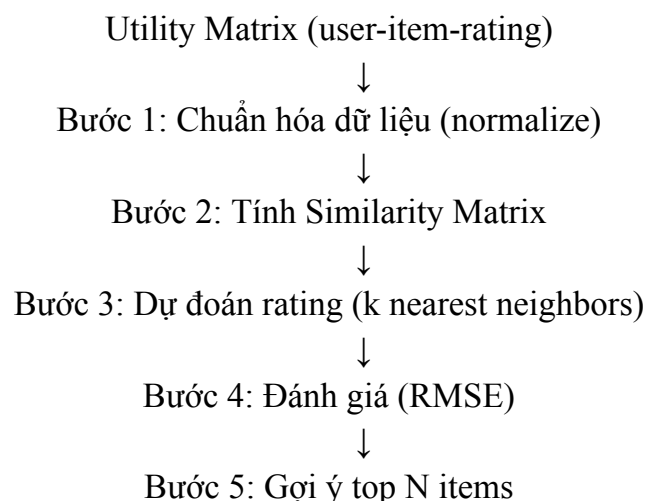
→ Tiết kiệm ~99% RAM với ma trận thưa.

Kết quả về việc chọn items nào để recommend cho mỗi user được thể hiện bởi các ô màu đỏ trong Hình 3d). Kết quả này có khác một chút so với kết quả tìm được bởi User-user CF ở 2 cột cuối cùng tương ứng với u5,u6. Đường như kết quả này hợp lý hơn vì từ Utility Matrix, có hai nhóm users thích hai nhóm items khác nhau.

Về mặt tính toán, Item-item CF có thể nhận được từ User-user CF bằng cách chuyển vị (transpose) ma trận utility, và coi như items đang rate users. Sau khi tính ra kết quả cuối cùng, ta lại chuyển vị một lần nữa để thu được kết quả.

Các bước chính:

Tổng quan flow



Bước 0 - Chuẩn bị dữ liệu

```
import pandas as pd  
import numpy as np  
from sklearn.metrics.pairwise import cosine_similarity  
from scipy import sparse  
from sklearn.model_selection import train_test_split
```

Dữ liệu đầu vào: mỗi dòng = 1 lượt tương tác
[user_id, item_id, rating]

Ví dụ với MyTV - tạo implicit rating từ duration:

```
interaction = (  
    df.groupby([  
        'vmp_movie_watch_history.member_id',  
        'vmp_movie_watch_history.content_id'  
    ])['vmp_movie_watch_history.duration']  
    .sum()  
    .reset_index()  
)  
interaction['score'] = np.log1p(  
    interaction['vmp_movie_watch_history.duration']  
)
```

Encode về 0-based index

```
user_ids = interaction['vmp_movie_watch_history.member_id'].unique()  
item_ids = interaction['vmp_movie_watch_history.content_id'].unique()
```

```
user2idx = {uid: idx for idx, uid in enumerate(user_ids)}  
item2idx = {iid: idx for idx, iid in enumerate(item_ids)}  
idx2item = {v: k for k, v in item2idx.items()}
```

```
interaction['user_idx'] = interaction[  
    'vmp_movie_watch_history.member_id'].map(user2idx)  
interaction['item_idx'] = interaction[  
    'vmp_movie_watch_history.content_id'].map(item2idx)
```

Tạo array [user_idx, item_idx, score]

```
Y_data = interaction[['user_idx', 'item_idx', 'score']].values
```

Train/Test split

```
Y_train, Y_test = train_test_split(Y_data, test_size=0.2, random_state=42)
```

```
print(f"Train: {Y_train.shape[0]:,} interactions")
```

```
print(f"Test: {Y_test.shape[0]:,} interactions")
```

```
print(f"Users: {len(user_ids):,}")
```

```
print(f"Items: {len(item_ids):,}")
```

Bước 1 - Class CF & Chuẩn hóa dữ liệu: Loại bỏ thiên lệch - user dễ tính vs khó tính.

```
class CF(object):
```

```
    def __init__(self, Y_data, k, dist_func=cosine_similarity, uuCF=1):
```

```
        """
```

```
        Y_data : array [user_id, item_id, rating]
```

```
        k      : số neighbor dùng để dự đoán
```

```
        uuCF   : 1 = User-User CF, 0 = Item-Item CF
```

```
        """
```

```
        self.uuCF = uuCF
```

```
        self.k     = k
```

```
        self.dist_func = dist_func
```

Item-Item CF: đổi cột user↔item

→ dùng lại thuật toán User-User trên ma trận chuyển vị

```
self.Y_data = Y_data if uuCF else Y_data[:, [1, 0, 2]]
```

```
self.n_users = int(np.max(self.Y_data[:, 0])) + 1
```

```
self.n_items = int(np.max(self.Y_data[:, 1])) + 1
```

```
self.Ybar_data = None
```

```
def normalize_Y(self):
```

```
    """
```

```
    Chuẩn hóa: trừ mỗi rating đi trung bình của user đó
```

```
    Mục đích: loại bỏ thiên lệch user dễ/khó tính
```

```
    """
```

```
    users = self.Y_data[:, 0]
```

```
    self.Ybar_data = self.Y_data.copy()
```

```
    self.mu = np.zeros((self.n_users,))
```

```
    for n in range(self.n_users):
```

```
        Lấy tất cả ratings của user n
```

```
        ids = np.where(users == n)[0].astype(np.int32)
```

```
        ratings = self.Y_data[ids, 2]
```

```
        Trung bình ratings của user n
```

```
        m = np.mean(ratings)
```

```
        if np.isnan(m):
```

```
m = 0
```

```
self.mu[n] = m
```

Trừ đi trung bình → normalized rating

```
self.Ybar_data[ids, 2] = ratings - m
```

Lưu dạng sparse matrix → tiết kiệm RAM
(phần lớn ma trận là 0 → không cần lưu hết)

```
self.Ybar = sparse.coo_matrix(  
    (self.Ybar_data[:, 2],  
     (self.Ybar_data[:, 1], self.Ybar_data[:, 0])),  
    (self.n_items, self.n_users)  
)  
self.Ybar = self.Ybar.tocsr()
```

Ví dụ minh họa normalize:

User An: ratings [5, 4, 1] → mean=3.33 → normalized [+1.67, +0.67, -2.33]

User Chi: ratings [1, 5, 4] → mean=3.33 → normalized [-2.33, +1.67, +0.67]

Dương (+) = thích hơn trung bình

Âm (-) = không thích bằng trung bình

0 = chưa xem (chưa biết)

Bước 2 - Tính Similarity Matrix: Đo độ giống nhau giữa từng cặp users (hoặc items).

```
def similarity(self):
```

```
    """
```

```
    Tính Similarity Matrix dùng Cosine Similarity
```

```
    S[i,j] = độ giống nhau giữa user i và user j
```

```
    Giá trị trong [-1, 1]:
```

```
    +1 = hoàn toàn giống nhau
```

```
    0 = không liên quan
```

```
    -1 = hoàn toàn ngược nhau
```

```
    """
```

```
    Ybar.T: mỗi CỘT = vector của 1 user
```

```
    cosine_similarity so sánh từng cặp cột
```

```
    self.S = self.dist_func(self.Ybar.T, self.Ybar.T)
```

```
def fit(self):
```

```
    """Train model: normalize + tính similarity"""
```

```
    self.normalize_Y()
```

```
self.similarity()
```

Ví dụ similarity matrix (7 users):

```
u0 u1 u2 u3 u4 u5 u6
u0 [ 1.00  0.83 -0.41 -0.10 -0.30  0.27 -0.25]
u1 [ 0.83  1.00 -0.43 -0.13 -0.31 -0.23 -0.28]
u2 [-0.41 -0.43  1.00  0.90  0.74  0.00  0.64]
```

...

→ u0 và u1 rất giống nhau (0.83)

→ u0 và u2 ngược nhau (-0.41)

Bước 3 - Dự đoán rating: Dùng k users giống nhất để dự đoán rating.

```
def __pred(self, u, i, normalized=1):
```

```
    """
```

```
    Dự đoán rating của user u cho item i
```

```
    Công thức: trung bình có trọng số của k neighbors
```

```
    """
```

```
    Bước 1: Tìm users đã xem item i
```

```
    ids = np.where(self.Y_data[:, 1] == i)[0]
```

```
    users Rated_i = self.Y_data[ids, 0].astype(np.int32)
```

```
    Bước 2: Similarity giữa u và những users đó
```

```
    sim = self.S[u, users Rated_i]
```

```
    Bước 3: Lấy k users có similarity cao nhất
```

```
    a = np.argsort(sim)[-self.k:]
```

```
    nearest_s = sim[a]
```

```
    Bước 4: Lấy normalized rating của k users cho item i
```

```
    r = self.Ybar[i, users Rated_i[a]]
```

```
    Bước 5: Trung bình có trọng số
```

```
    1e-8 tránh chia cho 0
```

```
    pred = (r * nearest_s)[0] / (np.abs(nearest_s).sum() + 1e-8)
```

```
    if normalized:
```

```
        return pred
```

```
    Denormalize: cộng lại trung bình của user u
```

```
    return pred + self.mu[u]
```

```

def pred(self, u, i, normalized=1):
    """
    Public API - thứ tự luôn là (user, item)
    User-User: gọi thẳng __pred(u, i)
    Item-Item: đổi u↔i vì data đã được transpose từ đầu
    """
    if self.uuCF:
        return self.__pred(u, i, normalized)
    return self.__pred(i, u, normalized) # Item-Item: đổi u và i

```

Ví dụ dự đoán An (u=0) cho Mật Danh (i=2), k=2:

Bước 1: Users đã xem Mật Danh = [Bình, Chi, Dung]

Bước 2: $\text{sim}(\text{An}, \text{Bình}) = +0.83$, $\text{sim}(\text{An}, \text{Chi}) = -0.40$, $\text{sim}(\text{An}, \text{Dung}) = -0.23$

Bước 3: $k=2 \rightarrow$ lấy Bình(+0.83) và Dung(-0.23)

Bước 4: normalized_rating Bình=+1.0, Dung=0

Bước 5: $(0.83 \times 1.0 + (-0.23) \times 0) / (0.83 + 0.23) = +0.78$

+ $\mu[\text{An}] = 3.33 \rightarrow 4.11$ sao $\rightarrow$ GỢI Ý!

Bước 4 - Đánh giá RMSE: Đo sai số trung bình của model.

```

def evaluate_RMSE(self, rate_test):
    """Tính RMSE trên tập test"""
    n_tests = rate_test.shape[0]
    SE = 0 # tổng bình phương sai số

    for n in range(n_tests):
        pred = self.pred(
            u=int(rate_test[n, 0]),
            i=int(rate_test[n, 1]),
            normalized=0 # cần denormalize để so sánh
        )
        SE += (pred - rate_test[n, 2]) ** 2

    return np.sqrt(SE / n_tests)

```

Chạy và so sánh 2 phương pháp:

User-User CF

```
rs_uu = CF(Y_train, k=30, uuCF=1)
```

```
rs_uu.fit()
```

```
rmse_uu = rs_uu.evaluate_RMSE(Y_test)
```

```
print(f"User-User CF, RMSE = {rmse_uu:.4f}")
```

Item-Item CF

```
rs_ii = CF(Y_train, k=30, uuCF=0)
rs_ii.fit()
rmse_ii = rs_ii.evaluate_RMSE(Y_test)
print(f"Item-Item CF, RMSE = {rmse_ii:.4f}")
```

Bước 5 - Gọi ý phim: Với mỗi user, tìm phim chưa xem có predicted rating dương.

```
def recommend(self, u):
    """
    Gọi ý items cho user u
    Chỉ gọi ý item có normalized predicted rating > 0
    (tức user có khả năng thích hơn trung bình)
    """
    Phim user u đã xem
    ids = np.where(self.Y_data[:, 0] == u)[0]
    items_watched = self.Y_data[ids, 1].tolist()
    recommended = []

    for i in range(self.n_items):
        if i not in items_watched:
            rating = self.__pred(u, i, normalized=1)
            if rating > 0:
                recommended.append((i, rating))

    Sort theo predicted rating giảm dần
    recommended.sort(key=lambda x: x[1], reverse=True)
    return recommended
```

```
def print_recommendation(self, top_n=10):
    """In gọi ý cho tất cả users"""
    for u in range(self.n_users):
        recs = self.recommend(u)[:top_n]
        print(f"User {u}: gọi ý {[r[0] for r in recs]}")
```

Ví dụ output:

User 0: gọi ý [2, 5, 8, ...]

User 1: gọi ý [1, 4, 7, ...]

User 2: gợi ý [] ← không có gợi ý phù hợp

Chạy toàn bộ

```
print("=" * 50)
print("USER-USER COLLABORATIVE FILTERING")
print("=" * 50)
rs_uu = CF(Y_train, k=30, uuCF=1)
rs_uu.fit()
rmse_uu = rs_uu.evaluate_RMSE(Y_test)
print(f"RMSE = {rmse_uu:.4f}")
rs_uu.print_recommendation(top_n=5)

print()
print("=" * 50)
print("ITEM-ITEM COLLABORATIVE FILTERING")
print("=" * 50)
rs_ii = CF(Y_train, k=30, uuCF=0)
rs_ii.fit()
rmse_ii = rs_ii.evaluate_RMSE(Y_test)
print(f"RMSE = {rmse_ii:.4f}")
rs_ii.print_recommendation(top_n=5)
```

Ưu điểm:

- Không cần metadata của item: chỉ dựa vào hành vi xem của users, phù hợp với các hệ thống thiếu thông tin mô tả nội dung như MyTV (chỉ 145/3,946 phim có category).
- Tận dụng được thông tin từ cộng đồng users: hành vi của những users giống nhau giúp dự đoán chính xác hơn, đặc biệt khi có nhiều users có sở thích tương đồng.
- Kết quả dễ giải thích, có thể nói rõ lý do gợi ý: "vì những người xem giống bạn cũng thích phim này".
- Không yêu cầu training phức tạp: Item-Item CF chỉ cần tính Similarity Matrix một lần, sau đó inference nhanh.

Nhược điểm:

- Cold start cho user mới, user chưa có lịch sử xem thì không tính được similarity với ai, không gợi ý được gì.

- Khó scale với dataset lớn, User-User CF phải tính $737,265 \times 737,265$ cặp similarity với MyTV $\rightarrow$ tốn RAM và thời gian tính toán cực lớn. Item-Item CF tốt hơn nhưng vẫn bị giới hạn.
- Ma trận Utility quá thưa, đa số users chỉ xem 3-8 phim (median = 3), vector similarity không đủ thông tin để so sánh chính xác.
- Similarity Matrix phải tính lại thường xuyên, mỗi khi có thêm ratings mới, đặc biệt với User-User CF, toàn bộ similarity cần được cập nhật lại.

- **Matrix Factorization Collaborative Filtering.**

Hướng tiếp cận khác cho Collaborative Filtering dựa trên Matrix Factorization (hoặc Matrix Decomposition), tức Phân tích ma trận thành nhân tử.

Nhắc lại rằng trong [Content-based Recommendation Systems](#), mỗi *item* được mô tả bằng một vector $\mathbf{x}$ được gọi là *item profile*. Trong phương pháp này, ta cần tìm một vector hệ số $\mathbf{w}$ tương ứng với mỗi *user* sao cho *rating* đã biết mà *user* đó cho *item* xấp xỉ với:

$$y \approx \mathbf{x}\mathbf{w}$$

Với cách làm trên, *Utility Matrix* $\mathbf{Y}$, giả sử đã được điền hết, sẽ xấp xỉ với:

$$\mathbf{Y} \approx \begin{bmatrix} \mathbf{x}_1\mathbf{w}_1 & \mathbf{x}_1\mathbf{w}_2 & \dots & \mathbf{x}_1\mathbf{w}_N \\ \mathbf{x}_2\mathbf{w}_1 & \mathbf{x}_2\mathbf{w}_2 & \dots & \mathbf{x}_2\mathbf{w}_N \\ \dots & \dots & \ddots & \dots \\ \mathbf{x}_M\mathbf{w}_1 & \mathbf{x}_M\mathbf{w}_2 & \dots & \mathbf{x}_M\mathbf{w}_N \end{bmatrix} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \dots \\ \mathbf{x}_M \end{bmatrix} [\mathbf{w}_1 \quad \mathbf{w}_2 \quad \dots \quad \mathbf{w}_N] = \mathbf{X}\mathbf{W}$$

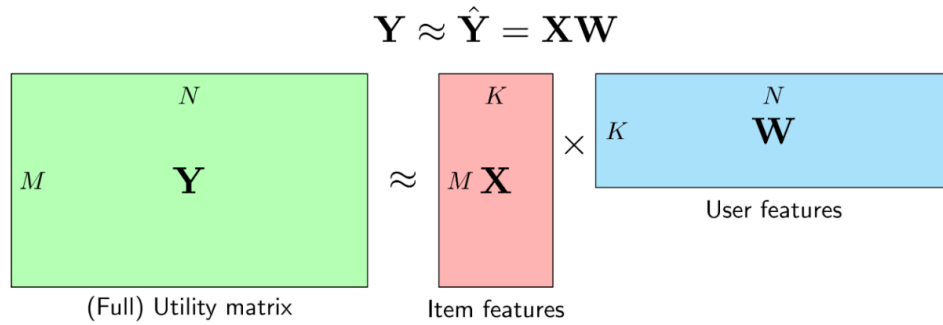
với M, N lần lượt là số *items* và số *users*.

Chú ý rằng, $\mathbf{x}$ được xây dựng dựa trên thông tin mô tả của *item* và quá trình xây dựng này độc lập với quá trình đi tìm hệ số phù hợp cho mỗi *user*. Như vậy, việc xây dựng *item profile* đóng vai trò rất quan trọng và có ảnh hưởng trực tiếp lên hiệu năng của mô hình. Thêm nữa, việc xây dựng từng mô hình riêng lẻ cho mỗi *user* dẫn đến kết quả chưa thực sự tốt vì không khai thác được đặc điểm của những *users* gần giống nhau.

Bây giờ, giả sử rằng ta không cần xây dựng từ trước các *item profile* $\mathbf{x}$ mà vector đặc trưng cho mỗi *item* này có thể được huấn luyện đồng thời với mô hình của mỗi *user* (ở đây là 1 vector hệ số). Điều này nghĩa là, biến số trong bài toán tối ưu là cả $\mathbf{X}$ và $\mathbf{W}$; trong đó, $\mathbf{X}$ là ma trận của toàn bộ *item profiles*, mỗi **hàng** tương ứng với 1 *item*, $\mathbf{W}$ là ma trận của toàn bộ *user models*, mỗi **cột** tương ứng với 1 *user*.

Với cách làm này, chúng ta đang cố gắng xấp xỉ *Utility Matrix* $\mathbf{Y} \in \mathbb{R}^{M \times N}$ bằng tích của hai ma trận $\mathbf{X} \in \mathbb{R}^{M \times K}$ và $\mathbf{W} \in \mathbb{R}^{K \times N}$.

Thông thường, K được chọn là một số nhỏ hơn rất nhiều so với M, N . Khi đó, cả hai ma trận $\mathbf{X}$ và $\mathbf{W}$ đều có rank không vượt quá K . Chính vì vậy, phương pháp này còn được gọi là *Low-Rank Matrix Factorization* (xem Hình 1).



Hình 1: Matrix Factorization. Utility matrix $\mathbf{Y}$ được phân tích thành tích của hai ma trận low-rank $\mathbf{X}$ và $\mathbf{W}$

Có một vài điểm lưu ý ở đây:

- Ý tưởng chính đằng sau Matrix Factorization cho Recommendation Systems là tồn tại các latent features (tính chất ẩn) mô tả sự liên quan giữa các items và users. Ví dụ với hệ thống gợi ý các bộ phim, tính chất ẩn có thể là hình sự, chính trị, hành động, hài, ...; cũng có thể là một sự kết hợp nào đó của các thể loại này; hoặc cũng có thể là bất cứ điều gì mà chúng ta không thực sự cần đặt tên. Mỗi item sẽ mang tính chất ẩn ở một mức độ nào đó tương ứng với các hệ số trong vector $\mathbf{x}$ của nó, hệ số càng cao tương ứng với việc mang tính chất đó càng cao. Tương tự, mỗi user cũng sẽ có xu hướng thích những tính chất ẩn nào đó và được mô tả bởi các hệ số trong vector $\mathbf{w}$ của nó. Hệ số cao tương ứng với việc user thích các bộ phim có tính chất ẩn đó. Giá trị của biểu thức $\mathbf{xw}$ sẽ cao nếu các thành phần tương ứng của $\mathbf{x}$ và $\mathbf{w}$ đều cao. Điều này nghĩa là item mang các tính chất ẩn mà user thích, vậy thì nên gợi ý item này cho user đó.
- Vậy tại sao Matrix Factorization lại được xếp vào Collaborative Filtering? Câu trả lời đến từ việc đi tối ưu hàm mất mát mà chúng ta sẽ thảo luận ở Mục 2. Về cơ bản, để tìm nghiệm của bài toán tối ưu, ta phải lần lượt đi tìm $\mathbf{X}$ và $\mathbf{W}$ khi thành phần còn lại được cố định. Như vậy, mỗi hàng của $\mathbf{X}$ sẽ phụ thuộc vào toàn bộ các cột của $\mathbf{W}$. Ngược lại, mỗi cột của $\mathbf{W}$ lại phụ thuộc vào toàn bộ các hàng của $\mathbf{X}$. Như vậy, có những mối quan hệ ràng buộc chặt chẽ giữa các thành phần của hai ma trận trên. Tức chúng ta cần sử dụng thông tin của tất cả để suy ra tất cả. Vậy nên phương pháp này cũng được xếp vào Collaborative Filtering.
- Trong các bài toán thực tế, số lượng items M và số lượng users N thường rất lớn. Việc tìm ra các mô hình đơn giản giúp dự đoán ratings cần được thực hiện một cách nhanh nhất có thể. Neighborhood-based Collaborative Filtering không yêu cầu việc learning quá nhiều, nhưng trong quá trình dự đoán

(inference), ta cần đi tìm độ similarity của user đang xét với toàn bộ các users còn lại rồi suy ra kết quả. Ngược lại, với Matrix Factorization, việc learning có thể hơi phức tạp một chút vì phải lặp đi lặp lại việc tối ưu một ma trận khi cố định ma trận còn lại, nhưng việc inference đơn giản hơn vì ta chỉ cần lấy tích của hai vector $\mathbf{x}_w$, mỗi vector có độ dài K là một số nhỏ hơn nhiều so với M, N . Vậy nên quá trình inference không yêu cầu khả năng tính toán cao. Việc này khiến nó phù hợp với các mô hình có tập dữ liệu lớn.

- Thêm nữa, việc lưu trữ hai ma trận $\mathbf{X}$ và $\mathbf{W}$ yêu cầu lượng bộ nhớ nhỏ khi so với việc lưu toàn bộ Similarity matrix trong Neighborhood-based Collaborative Filtering. Cụ thể, ta cần bộ nhớ để chứa $K(M+N)$ phần tử thay vì lưu M^2 hoặc N^2 của Similarity matrix.

Xây dựng và tối ưu hàm mất mát

Hàm mất mát

Tương tự như trong [Content-based Recommendation Systems](#), việc xây dựng hàm mất mát cũng được dựa trên các thành phần đã được đề cập của Utility Matrix $\mathbf{Y}$, có khác một chút là không có thành phần bias và biến tối ưu là cả $\mathbf{X}$ và $\mathbf{W}$. Việc thêm bias vào sẽ được thảo luận ở Mục 4. Việc xây dựng hàm mất mát cho Matrix Factorization là tương đối dễ hiểu:

$$\mathcal{L}(\mathbf{X}, \mathbf{W}) = \frac{1}{2s} \sum_{n=1}^N \sum_{m:r_{mn}=1} (y_{mn} - \mathbf{x}_m \mathbf{w}_n)^2 + \frac{\lambda}{2} (\|\mathbf{X}\|_F^2 + \|\mathbf{W}\|_F^2) \quad (1)$$

trong đó $r_{mn} = 1$ nếu item thứ m đã được đánh giá bởi user thứ n , $\|\bullet\|_F^2$ là [Frobenius norm](#), tức căn bậc hai của tổng bình phương tất cả các phần tử của ma trận (giống với norm 2 trong vector), s là toàn bộ số ratings đã có. Thành phần thứ nhất chính là trung bình sai số của mô hình. Thành phần thứ hai trong hàm mất mát phía trên là [l₂ regularization](#), giúp tránh [overfitting](#).

Lưu ý: Giá trị ratings thường là các giá trị đã được chuẩn hoá, bằng cách trừ mỗi hàng của Utility Matrix đi trung bình cộng của các giá trị đã biết của hàng đó (item-based) hoặc trừ mỗi cột đi trung bình cộng của các giá trị đã biết trong cột đó (user-based). Trong một số trường hợp nhất định, ta không cần chuẩn hoá ma trận này, nhưng kèm theo đó phải có thêm các kỹ thuật khác để giải quyết vấn đề *thiên lệch* trong khi rating.

Việc tối ưu đồng thời $\mathbf{X}, \mathbf{W}$ là tương đối phức tạp, thay vào đó, phương pháp được sử dụng là lần lượt tối ưu một ma trận trong khi cố định ma trận kia, tới khi hội tụ.

Tối ưu hàm mất mát

Khi cố định $\mathbf{X}$, việc tối ưu $\mathbf{W}$ chính là bài toán tối ưu trong Content-based Recommendation Systems:

$$\mathcal{L}(\mathbf{W}) = \frac{1}{2s} \sum_{n=1}^N \sum_{m:r_{mn}=1} (y_{mn} - \mathbf{x}_m \mathbf{w}_n)^2 + \frac{\lambda}{2} \|\mathbf{W}\|_F^2 \quad (2)$$

Khi cố định $\mathbf{W}$, việc tối ưu $\mathbf{X}$ được đưa về tối ưu hàm:

$$\mathcal{L}(\mathbf{X}) = \frac{1}{2s} \sum_{n=1}^N \sum_{m:r_{mn}=1} (y_{mn} - \mathbf{x}_m \mathbf{w}_n)^2 + \frac{\lambda}{2} \|\mathbf{X}\|_F^2 \quad (3)$$

Hai bài toán này sẽ được tối ưu bằng [Gradient Descent](#).

Chúng ta có thể thấy rằng bài toán (2) có thể được tách thành N bài toán nhỏ, mỗi bài toán ứng với việc đi tối ưu một cột của ma trận $\mathbf{W}$:

$$\mathcal{L}(\mathbf{w}_n) = \frac{1}{2s} \sum_{m:r_{mn}=1} (y_{mn} - \mathbf{x}_m \mathbf{w}_n)^2 + \frac{\lambda}{2} \|\mathbf{w}_n\|_2^2 \quad (4)$$

Vì biểu thức trong dấu $\sum$ chỉ phụ thuộc vào các *items* đã được *rated* bởi *user* đang xét, ta có thể đơn giản nó bằng cách đặt $\hat{\mathbf{X}}_n$ là ma trận được tạo bởi các hàng tương ứng với các *items* đã được *rated* đó, và $\hat{\mathbf{y}}_n$ là các *ratings* tương ứng. Khi đó:

$$\mathcal{L}(\mathbf{w}_n) = \frac{1}{2s} \|\hat{\mathbf{y}}_n - \hat{\mathbf{X}}_n \mathbf{w}_n\|^2 + \frac{\lambda}{2} \|\mathbf{w}_n\|_2^2 \quad (5)$$

và đạo hàm của nó:

$$\frac{\partial \mathcal{L}(\mathbf{w}_n)}{\partial \mathbf{w}_n} = -\frac{1}{s} \hat{\mathbf{X}}_n^T (\hat{\mathbf{y}}_n - \hat{\mathbf{X}}_n \mathbf{w}_n) + \lambda \mathbf{w}_n \quad (6)$$

Vậy công thức cập nhật cho mỗi cột của $\mathbf{W}$ là:

$$\mathbf{w}_n = \mathbf{w}_n - \eta \left(-\frac{1}{s} \hat{\mathbf{X}}_n^T (\hat{\mathbf{y}}_n - \hat{\mathbf{X}}_n \mathbf{w}_n) + \lambda \mathbf{w}_n \right) \quad (7)$$

Tương tự như thế, mỗi cột của $\mathbf{X}$, tức vector cho mỗi *item*, sẽ được tìm bằng cách tối ưu:

$$\mathcal{L}(\mathbf{x}_m) = \frac{1}{2s} \sum_{n:r_{mn}=1} (y_{mn} - \mathbf{x}_m \mathbf{w}_n)^2 + \frac{\lambda}{2} \|\mathbf{x}_m\|_2^2 \quad (8)$$

Đặt $\hat{\mathbf{W}}_m$ là ma trận được tạo bằng các cột của $\mathbf{W}$ ứng với các *users* đã đánh giá *item* đó và $\hat{\mathbf{y}}^m$ là vector *ratings* tương ứng. (8) trở thành:

$$\mathcal{L}(\mathbf{x}_m) = \frac{1}{2s} \|\hat{\mathbf{y}}^m - \mathbf{x}_m \hat{\mathbf{W}}_m\|^2 + \frac{\lambda}{2} \|\mathbf{x}_m\|_2^2 \quad (9)$$

Tương tự như trên, công thức cập nhật cho mỗi hàng của $\mathbf{X}$ sẽ có dạng:

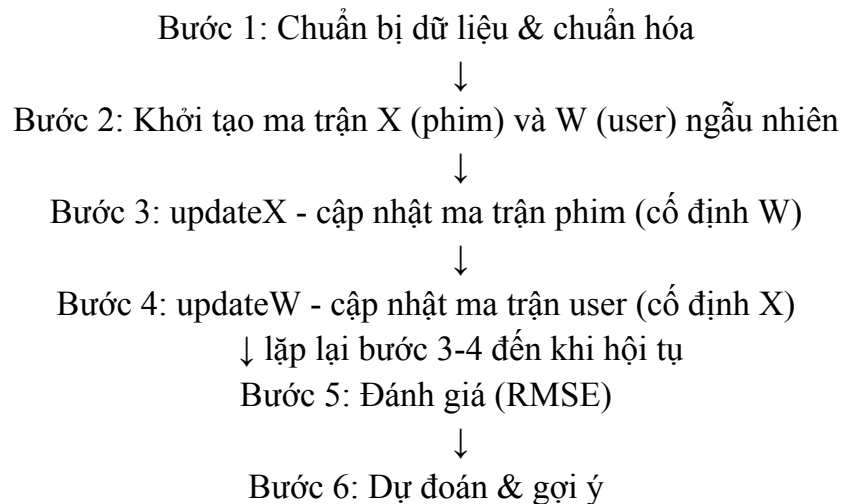
$$\mathbf{x}_m = \mathbf{x}_m - \eta \left(-\frac{1}{s} (\hat{\mathbf{y}}^m - \mathbf{x}_m \hat{\mathbf{W}}_m) \hat{\mathbf{W}}_m^T + \lambda \mathbf{x}_m \right) \quad (10)$$

Các bước chính:

Tổng quan flow

Utility Matrix (user-item-rating)





Bước 0 - Chuẩn bị dữ liệu

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
```

Dữ liệu đầu vào: [user_idx, item_idx, score]

Với MyTV - tạo implicit rating từ duration:

```
interaction = (
    df.groupby([
        'vmp_movie_watch_history.member_id',
        'vmp_movie_watch_history.content_id'
    ])['vmp_movie_watch_history.duration']
    .sum()
    .reset_index()
)
```

Implicit rating: xem lâu hơn = thích hơn

log1p để giảm chênh lệch giữa xem ít và xem nhiều

```
interaction['score'] = np.log1p(
    interaction['vmp_movie_watch_history.duration']
)
```

Encode về 0-based index

(model cần index liên tục từ 0, không phải member_id gốc)

```
user_ids = interaction['vmp_movie_watch_history.member_id'].unique()
item_ids = interaction['vmp_movie_watch_history.content_id'].unique()
```

```
user2idx = {uid: idx for idx, uid in enumerate(user_ids)}
```

```

item2idx = {iid: idx for idx, iid in enumerate(item_ids)}
idx2user = {v: k for k, v in user2idx.items()}
idx2item = {v: k for k, v in item2idx.items()}

```

```

interaction['user_idx'] = interaction[
    'vmp_movie_watch_history.member_id'].map(user2idx)
interaction['item_idx'] = interaction[
    'vmp_movie_watch_history.content_id'].map(item2idx)

```

Tạo array [user_idx, item_idx, score]

```

Y_data = interaction[['user_idx', 'item_idx', 'score']].values

```

Train/Test split

```

Y_train, Y_test = train_test_split(Y_data, test_size=0.2, random_state=42)

```

```

print(f"Train: {len(Y_train):,} interactions")
print(f"Test: {len(Y_test):,} interactions")
print(f"Users: {len(user_ids):,}")
print(f"Items: {len(item_ids):,}")
print(f"Score range: {Y_data[:,2].min():.2f} → {Y_data[:,2].max():.2f}")

```

Bước 1 - Class MF & Chuẩn hóa

```

class MF(object):
    def __init__(self, Y_data, K, lam=0.1, Xinit=None, Winit=None,
                  learning_rate=0.5, max_iter=100,
                  print_every=10, user_based=1):
        """
        Y_data      : array [user_idx, item_idx, score]
        K           : số latent features (chiều ẩn)
        lam         :  $\lambda$  regularization - tránh overfitting
        learning_rate :  $\eta$  - bước học trong gradient descent
        max_iter     : số vòng lặp tối đa
        user_based   : 1=chuẩn hóa theo user, 0=theo item
        """
        self.Y_raw_data = Y_data
        self.K           = K
        self.lam         = lam
        self.learning_rate = learning_rate
        self.max_iter     = max_iter
        self.print_every  = print_every

```

```

self.user_based = user_based
self.n_users = int(np.max(Y_data[:, 0])) + 1
self.n_items = int(np.max(Y_data[:, 1])) + 1
self.n_ratings = Y_data.shape[0]

```

Khởi tạo X và W ngẫu nhiên

X: ma trận đặc trưng phim ($n_items \times K$)

W: ma trận sở thích user ($K \times n_users$)

```

self.X = np.random.randn(self.n_items, K) \
    if Xinit is None else Xinit
self.W = np.random.randn(K, self.n_users) \
    if Winit is None else Winit

```

```

self.Y_data_n = self.Y_raw_data.copy()

```

```

def normalize_Y(self):

```

```

    """

```

Chuẩn hóa rating: trừ đi trung bình của user/item

Mục đích: loại bỏ thiên lệch user dễ/khó tính

```

    """

```

```

    if self.user_based:

```

```

        user_col, item_col = 0, 1

```

```

        n_objects = self.n_users

```

```

    else:

```

```

        user_col, item_col = 1, 0

```

```

        n_objects = self.n_items

```

```

    users = self.Y_raw_data[:, user_col]

```

```

    self.mu = np.zeros((n_objects,))

```

```

    for n in range(n_objects):

```

```

        ids = np.where(users == n)[0].astype(np.int32)

```

```

        ratings = self.Y_data_n[ids, 2]

```

```

        m = np.mean(ratings)

```

```

        if np.isnan(m):

```

```

            m = 0

```

```

        self.mu[n] = m # lưu mean để dùng khi predict

```

```

        self.Y_data_n[ids, 2] = ratings - m # normalize

```

Ví dụ normalize:

User An: ratings [7.2, 6.8, 1.4] $\rightarrow$ mean=5.13

normalized [+2.07, +1.67, -3.73]

Dương (+) = xem nhiều hơn trung bình $\rightarrow$ thích

Âm (-) = xem ít hơn trung bình $\rightarrow$ không thích

Lưu $\mu[An]=5.13 \rightarrow$ cộng lại khi predict

Bước 2 - Hàm mất mát (Loss)

```
def loss(self):
```

```
    """
```

```
    L = trung_bình(score_thật - score_dự_đoán)2
```

```
    +  $\lambda \times (||X||^2 + ||W||^2)$ 
```

```
    Phần 1: đo sai số trung bình
```

```
    Phần 2: regularization  $\rightarrow$  phạt nếu X,W quá lớn
```

```
    """
```

```
    L = 0
```

```
    for i in range(self.n_ratings):
```

```
        n = int(self.Y_data_n[i, 0]) # user index
```

```
        m = int(self.Y_data_n[i, 1]) # item index
```

```
        rate = self.Y_data_n[i, 2] # normalized rating
```

```
        score_dự_đoán = X[m,:] · W[:,n]
```

```
        L += 0.5 * (rate - self.X[m, :].dot(self.W[:, n])) ** 2
```

```
    L /= self.n_ratings # trung bình
```

```
    regularization
```

```
    L += 0.5 * self.lam * (
```

```
        np.linalg.norm(self.X, 'fro') +
```

```
        np.linalg.norm(self.W, 'fro')
```

```
    )
```

```
    return L
```

Loss giảm dần theo iteration = model đang học tốt

Loss không giảm nữa = model đã hội tụ

Bước 3 - updateX (Cập nhật ma trận phim)

```
def get_users_who_rate_item(self, item_id):
```

```
    """Lấy tất cả users đã xem item_id và ratings tương ứng"""
```

```
    ids = np.where(self.Y_data_n[:, 1] == item_id)[0]
```

```
    user_ids = self.Y_data_n[ids, 0].astype(np.int32)
```

```
    ratings = self.Y_data_n[ids, 2]
```

```
return user_ids, ratings
```

```
def updateX(self):
```

```
    """
```

```
    Cập nhật ma trận phim X - cố định W
```

```
    Với mỗi phim m: tìm X[m,:] tốt nhất
```

```
    """
```

```
    for m in range(self.n_items):
```

```
        Lấy users đã xem phim m
```

```
        user_ids, ratings = self.get_users_who_rate_item(m)
```

```
        if len(user_ids) == 0:
```

```
            continue
```

```
        W của những users đó
```

```
        Wm = self.W[:, user_ids]
```

```
        Tính sai số
```

```
        error = ratings - self.X[m, :].dot(Wm)
```

```
        Tính gradient (hướng tăng của loss)
```

```
        grad_xm = -error.dot(Wm.T) / self.n_ratings \
```

```
                + self.lam * self.X[m, :]
```

```
        Cập nhật: đi ngược gradient → giảm loss
```

```
        self.X[m, :] -= self.learning_rate * grad_xm.reshape((self.K,))
```

Ví dụ:

Phim "Đấu Phá" được xem bởi An, Bình (thích) và Chi (không thích)

→ updateX điều chỉnh X[Đấu_Phá] để:

$X[\text{Đấu_Phá}] \cdot W[\text{An}] \rightarrow \text{cao}$ (An thích)

$X[\text{Đấu_Phá}] \cdot W[\text{Bình}] \rightarrow \text{cao}$ (Bình thích)

$X[\text{Đấu_Phá}] \cdot W[\text{Chi}] \rightarrow \text{thấp}$ (Chi không thích)

Bước 4 - updateW (Cập nhật ma trận user)

```
def get_items Rated_by_user(self, user_id):
```

```
    """Lấy tất cả phim đã xem bởi user_id và ratings tương ứng"""
```

```
    ids = np.where(self.Y_data_n[:, 0] == user_id)[0]
```

```
    item_ids = self.Y_data_n[ids, 1].astype(np.int32)
```

```
    ratings = self.Y_data_n[ids, 2]
```

```
    return item_ids, ratings
```

```
def updateW(self):
```

```
    """
```

```
    Cập nhật ma trận user W - cố định X
```

```
    Với mỗi user n: tìm W[:,n] tốt nhất
```

```
    """
```

```
    for n in range(self.n_users):
```

```
        Lấy phim user n đã xem
```

```
        item_ids, ratings = self.get_itemsRatedByUser(n)
```

```
        if len(item_ids) == 0:
```

```
            continue
```

```
        X của những phim đó
```

```
        Xn = self.X[item_ids, :]
```

```
        Tính sai số
```

```
        error = ratings - Xn.dot(self.W[:, n])
```

```
        Tính gradient
```

```
        grad_wn = -Xn.T.dot(error) / self.n_ratings \
```

```
            + self.lam * self.W[:, n]
```

```
        Cập nhật
```

```
        self.W[:, n] -= self.learning_rate * grad_wn.reshape((self.K,))
```

Ví dụ:

User An đã xem Đấu Phá(+), Mị Nguyệt(+), Doraemon(-)

→ updateW điều chỉnh W[An] để:

$X[\text{Đấu_Phá}] \cdot W[\text{An}] \rightarrow \text{cao}$

$X[\text{Mị_Nguyệt}] \cdot W[\text{An}] \rightarrow \text{cao}$

$X[\text{Doraemon}] \cdot W[\text{An}] \rightarrow \text{thấp}$

Bước 5 - fit (Vòng lặp chính)

```
def fit(self):
```

```
    """
```

```
    Train model: luân phiên updateX và updateW
```

```
    đến khi loss hội tụ
```

```
    """
```

```
    self.normalize_Y()    chuẩn hóa 1 lần đầu
```

```

    for it in range(self.max_iter):
        self.updateX()    cố định W → tối ưu X
        self.updateW()    cố định X → tối ưu W

    if (it + 1) % self.print_every == 0:
        rmse = self.evaluate_RMSE(self.Y_raw_data)
        print(f"iter={it+1:3d}, "
              f"loss={self.loss():.4f}, "
              f"RMSE train={rmse:.4f}")

```

Kết quả kỳ vọng:

iter= 10, loss=5.67, RMSE train=1.20 ← mới học

iter= 50, loss=0.48, RMSE train=1.03 ← đang học

iter=100, loss=0.26, RMSE train=1.03 ← hội tụ

- + Loss giảm dần → model đang học đúng hướng
- + RMSE ổn định → không cần train thêm

Bước 6 - Đánh giá RMSE

```

def evaluate_RMSE(self, rate_test):
    """
    RMSE = căn(trung bình(score_thật - score_dự_đoán)2)
    Đo sai số trung bình của model
    """
    n_tests = rate_test.shape[0]
    SE = 0

    for n in range(n_tests):
        pred = self.pred(rate_test[n, 0], rate_test[n, 1])
        SE += (pred - rate_test[n, 2]) ** 2

    return np.sqrt(SE / n_tests)

```

Chạy và so sánh User-based vs Item-based:

```

print("Training User-based MF...")
rs_user = MF(Y_train, K=10, lam=0.1,
             learning_rate=0.5, max_iter=50,
             print_every=10, user_based=1)
rs_user.fit()
rmse_user = rs_user.evaluate_RMSE(Y_test)

```

```
print(f"User-based MF, RMSE = {rmse_user:.4f}")
```

```
print("\nTraining Item-based MF...")
```

```
rs_item = MF(Y_train, K=10, lam=0.1,  
             learning_rate=0.5, max_iter=50,  
             print_every=10, user_based=0)
```

```
rs_item.fit()
```

```
rmse_item = rs_item.evaluate_RMSE(Y_test)
```

```
print(f"Item-based MF, RMSE = {rmse_item:.4f}")
```

Kết quả tham khảo (MovieLens 100k):

User-based MF: RMSE = 1.0604

Item-based MF: RMSE = 1.0498 ← tốt hơn một chút

Bước 7 - Dự đoán & Gợi ý

```
def pred(self, u, i):
```

```
    """Dự đoán score của user u cho item i"""
```

```
    u, i = int(u), int(i)
```

```
    # Cộng lại mean để denormalize
```

```
    bias = self.mu[u] if self.user_based else self.mu[i]
```

```
    return self.X[i, :].dot(self.W[:, u]) + bias
```

```
def recommend(self, user_id, top_n=10):
```

```
    """Gợi ý top N phim cho user"""
```

```
    Phim đã xem
```

```
    item_ids, _ = self.get_items Rated_by_user(user_id)
```

```
    watched = set(item_ids.tolist())
```

```
    Dự đoán tất cả phim chưa xem
```

```
    bias = self.mu[user_id] if self.user_based else 0
```

```
    scores = self.X.dot(self.W[:, user_id]) + bias
```

```
    recs = [
```

```
        (i, scores[i])
```

```
        for i in range(self.n_items)
```

```
        if i not in watched
```

```
    ]
```

```
    Sort theo score giảm dần
```

```
    recs.sort(key=lambda x: x[1], reverse=True)
```

```
return recs[:top_n]
```

Ví dụ gợi ý cho 1 user:

```
target_user_orig = 5759993
```

```
target_user_idx = user2idx.get(target_user_orig)
```

```
top10 = rs_item.recommend(target_user_idx, top_n=10)
```

Map content_id về tên phim

```
name_map = df.groupby(
```

```
    'vmp_movie_watch_history.content_id'
```

```
)['vmp_movie_watch_history.content_name'].first().to_dict()
```

```
print(f"Top 10 gợi ý cho user {target_user_orig}:")
```

```
results = pd.DataFrame([ {
```

```
    'content_id': idx2item[item_idx],
```

```
    'content_name': name_map.get(idx2item[item_idx], 'Unknown'),
```

```
    'pred_score': round(score, 4)
```

```
    } for item_idx, score in top10])
```

```
print(results.to_string(index=False))
```

LOSS:

Loss = sai_số + $\lambda \times \text{độ_lớn}(w)$

→ Dùng để train model

→ Có thêm phần regularization

→ Model tối ưu loss để học

RMSE:

RMSE = căn(trung_bình(dự_đoán - thật)²)

→ Dùng để đánh giá model

→ Không có regularization

→ Đo sai số thực tế trên test data

Ưu điểm:

- Scale tốt với dataset lớn, chỉ cần lưu 2 ma trận nhỏ X (phim $\times$ K) và W (K $\times$ user) thay vì Similarity Matrix khổng lồ. Với MyTV: K=10 chỉ cần 7.4 triệu phần tử thay vì 543 tỷ phần tử của User-User CF.
- Inference cực nhanh, dự đoán score chỉ là 1 phép dot product: X[phim] $\cdot$ W[user], phù hợp với hệ thống real-time.

- Tự động học latent features ẩn, model tự phát hiện các đặc trưng ngầm (ví dụ: phim hành động Trung Quốc được xem bởi nhóm tuổi trung niên) mà không cần metadata hay đặt tên thủ công.
- Hiệu quả hơn với dataset lớn, càng nhiều dữ liệu thì X và W càng được học chính xác hơn, phù hợp với 24M records của MyTV.
- Linh hoạt, dễ dàng thêm bias, regularization, hoặc kết hợp với các kỹ thuật khác (Content-Based $\rightarrow$ Hybrid System).

Nhược điểm:

- Training chậm, phải lặp đi lặp lại update X và update W nhiều lần đến khi hội tụ, tốn thời gian hơn NCF.
- Khó giải thích kết quả, latent features không có tên cụ thể, không thể nói rõ tại sao gợi ý phim này cho user đó.
- Cold start cho user và item mới, user hoặc phim mới chưa có trong training data thì không có vector X hay W tương ứng $\rightarrow$ không gợi ý được.
- Cần tuning hyperparameter, K (số latent features), λ (regularization), learning rate cần được chọn cẩn thận, ảnh hưởng lớn đến kết quả.
- Implicit rating chưa chắc chính xác, với MyTV không có rating thật, phải tạo score từ duration ($\log 1p$) $\rightarrow$ giả định xem lâu = thích có thể sai trong một số trường hợp (để phim chạy nền, buffer time...).

D. Hybrid recommendation system

Kết hợp cả 2 dạng Content-based và Collaborative.

E. Các thuật toán sử dụng

Như mọi mô hình học khác việc tối thiểu hàm mất mát luôn là việc cần thiết để tăng độ chính xác cho mô hình. Để tối ưu hàm mất mát bên trên có 2 phương pháp hay dùng là Stochastic gradient descent và Alternating least squares.

1. Phương pháp Stochastic gradient descent (SGD):

Đầu tiên Gradient descent (GD) là gì? Quay lại hồi cấp 3 khi muốn tìm điểm cực tiểu toàn cục của 1 hàm số ta sẽ tìm đạo hàm và giải phương trình bằng 0, sau đó thay các nghiệm tìm được và so sánh các cực tiểu cục bộ. Nhưng đối với hàm mất mát với đạo hàm phức tạp thì ngay cả việc tìm nghiệm bằng 0 cũng không hề dễ dàng. Và GD là phương pháp giúp chúng ta tìm được điểm tiệm cận cực tiểu đó, bằng cách chọn 1 điểm bất kỳ, sau đó lặp lại 1 công thức giúp tiến dần điểm đó đến khi đạo hàm gần bằng 0 tức điểm cực tiểu. Chính vì vậy GD được gọi là giảm đạo hàm hay phương pháp xuống dốc. Cho x_t là điểm hiện tại, thì điểm tiếp theo x_{t+1} sẽ được tính bằng $x_t - \eta$ lần đạo hàm tại x_t với η (đọc là eta) là hệ số thể hiện tốc độ di chuyển của điểm

$$x_{t+1} = x_t - \eta f'(x_t)$$

Để giải thích công thức này thì hơi dài dòng nên có thể hiểu đơn giản là: sau mỗi lần lặp điểm hiện tại sẽ giảm (hoặc tăng nếu ở bên trái cực tiểu) 1 lượng tùy vào tốc độ eta và đạo hàm đến khi giá trị của đạo hàm gần bằng 0. Công thức đối với hàm vecto như hàm mất mát cũng sẽ tương tự:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} f(\theta_t)$$

Với θ là vecto hay điểm dữ liệu $\nabla_{\theta} f(\theta_t)$ là đạo hàm tại vecto hiện tại (∇ đọc là nabra)

SGD - giảm đạo hàm ngẫu nhiên là một phương pháp nhằm cải thiện tốc độ tìm điểm cực tiểu so với khi dùng GD. Khi dùng GD trong mỗi vòng lặp ta chỉ tính đạo hàm tại 1 điểm hiện tại rồi cập nhật điểm này dựa trên đạo hàm đó, còn SGD thì tính đạo hàm tại toàn bộ các điểm dữ liệu sau đó giảm theo đạo hàm của 1 điểm ngẫu nhiên. Và sự ngẫu nhiên này sẽ cho cơ hội nhảy 1 bước xa khi tính toán giúp tốc độ tiến đến điểm cần tìm nhanh hơn (do tốn ít vòng lặp hơn GD). Vì vậy đối với bài toán có lượng dữ liệu lớn như bài toán này thì SGD là phương pháp rất hiệu quả. Khi đó công thức cập nhật sẽ là:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} f(\theta, x_i, y_i)$$

Đặt $e_{ui} \stackrel{\text{def}}{=} r_{ui} - q_i^T p_u$. Khi áp dụng vào hàm mất mát (2) bên trên ta sẽ có công thức cập nhật của từng biến:

$$q_i \leftarrow q_i + \gamma(e_{ui} p_u - \lambda q_i)$$

$$p_u \leftarrow p_u + \gamma(e_{ui} q_i - \lambda p_u)$$

Ta chọn 2 điểm bắt đầu cho 2 biến số cần tìm rồi giảm dần theo đạo hàm để có được điểm cực tiểu (điểm khiến hàm mất mát đạt giá trị nhỏ nhất) chính là điểm cần tìm.

2. Phương pháp Alternating least squares (ALS):

Tuy nhiên trong một vài trường hợp sử dụng phương pháp ALS lại có hiệu quả hơn. ALS hay còn gọi là phương pháp bình phương tối thiểu thay phiên nhau. Gọi là thay phiên nhau bởi hàm mất mát trên có 2 biến khiến hàm không lồi (khó để tìm cực tiểu) chính vì vậy ta thay phiên cố định 1 biến rồi tối ưu hàm theo biến còn lại. Khi cố định 1 biến hàm trở thành hàm bậc 2 và có thể giải bằng phương pháp bình phương tối thiểu. Việc cố định 1 biến rồi tính toán lại biến còn lại được thực hiện lần lượt cho đến khi các điểm hội tụ lại tại điểm cực tiểu.

Tựu chung thì SGD dễ sử dụng và tốc độ nhanh hơn ALS tuy nhiên có 2 trường hợp mà ALS lại được ưu tiên hơn.

Trường hợp 1 là hệ thống có khả năng chạy các tiến trình 1 cách song song.
Ta thấy khi cố định 1 trong 2 cột công thức hàm cần tối ưu sẽ trở thành

$$\sum_{(u,i) \in K} (r_{ui} - q_i^T p_u)^2 + \lambda \|q_i\|^2$$

Từ đó bài toán có thể chuyển thành việc tối ưu từng cột 1 của q (hoặc p) cho nên việc sử dụng trong hệ thống song song sẽ dễ dàng hơn. Trong khi nếu SGD sử dụng trong hệ thống song song sẽ dễ gặp vấn đề bị ghi đè nếu kết quả cập nhật có liên quan đến 2 phép toán diễn ra song song.

Trường hợp thứ 2 là khi sử dụng với hệ thống tập trung vào các dữ liệu ẩn. Bởi không phải lúc nào tập huấn luyện cũng thừa thớt, nên việc lặp lại qua tất cả các điểm dữ liệu theo cách của SGD lại trở lên không thực tiễn. Trong trường hợp này sử dụng ALS cũng có hiệu quả cao hơn.

Nguồn tham khảo:

<https://machinelearningcoban.com/2017/05/17/contentbasedrecommendersys/>

<https://machinelearningcoban.com/2017/05/24/collaborativefiltering/>

<https://machinelearningcoban.com/2017/05/31/matrixfactorization/>

<https://viblo.asia/p/cong-nghe-matrix-factorization-cho-he-thong-goi-y-naQZRJe0Zvx>